

Homework 4: RNN Models for SLU and Recursive Time-Series Forecasting

Taha Majlesi - 810101504
Department of Electrical and Computer Engineering
University of Tehran
Email: student@example.com

Abstract—This report documents a complete solution to Homework 4: (1) RNN-based Spoken Language Understanding (SLU) on ATIS (slot filling and intent detection) and (2) recursive time-series forecasting of the S&P 500. The document includes: exact preprocessing steps and design choices, full model descriptions and training details, metric definitions, analysis and answers to the theory questions (stationarity, cyclical encoding, ergodicity, decoding behavior), reproducible instructions, and placeholders for results. All figures used by the report are saved under ‘codes/images/’ and tables exported by notebooks are provided as LaTeX fragments to be included directly in this document.

I. Introduction

This comprehensive assignment explores two fundamental challenges in sequence modeling that have significant real-world applications and theoretical importance in machine learning. The first task addresses Spoken Language Understanding (SLU), a critical component of modern conversational AI systems such as virtual assistants, automated customer service platforms, and dialogue systems. SLU encompasses two interdependent sub-tasks: intent detection, which classifies the user’s overarching goal or intention, and slot filling, which extracts structured semantic information from utterances using the BIO (Beginning, Inside, Outside) tagging scheme.

The second task focuses on financial time-series forecasting, specifically predicting the S&P 500 index—a benchmark indicator of U.S. stock market performance. This problem represents a canonical example of challenging time-series prediction where traditional statistical methods often struggle due to non-stationarity, volatility clustering, and complex temporal dependencies. The assignment requires implementing both classical approaches (MLP baseline) and modern deep learning architectures (CNN+LSTM, CNN+GRU, TCN) to capture nonlinear patterns and long-range dependencies.

Beyond mere implementation, this assignment demands a deep theoretical understanding of sequence modeling principles, careful experimental design to ensure reproducible results, and rigorous evaluation methodologies. We implement multiple architectural variants for each task—BiRNN baseline, BiLSTM joint model, and Encoder-Decoder non-aligned for SLU; MLP, CNN+LSTM, CNN+GRU, and TCN for time-series forecasting—while maintaining strict controls for fair comparison. Particular

attention is given to data preprocessing, hyperparameter optimization, model validation strategies, and comprehensive performance evaluation using task-appropriate metrics.

The report provides complete theoretical foundations, mathematical derivations, implementation details, experimental results, and critical analysis of the strengths and limitations of each approach. All code is designed to be reproducible, well-documented, and easily extensible for future research.

II. Question 1: RNN Models for SLU (Detailed Answers and Complete Solutions)

A. 1.1 Data Preparation (12 points; full exposition)

1) **Exploration and dataset statistics:** The initial phase of any machine learning project involves thorough dataset characterization to understand the problem complexity, identify potential challenges, and establish baseline expectations. For the ATIS dataset, we begin with systematic exploration of the raw data structure, statistical properties, and label distributions.

The ATIS (Airline Travel Information System) dataset represents real user queries to an automated airline reservation system, making it ecologically valid for SLU research. Each example consists of a natural language utterance, its corresponding intent label, and a sequence of BIO-tagged slot labels that extract structured information such as departure cities, arrival cities, dates, and airline preferences.

Our exploration pipeline implements several critical analyses:

- 1) **Sample Count Verification:** Ensures consistent split sizes across different dataset versions and preprocessing stages.
- 2) **Vocabulary Analysis:** Examines word frequency distributions, out-of-vocabulary rates, and potential subword tokenization benefits.
- 3) **Sequence Length Statistics:** Computes mean, median, and maximum utterance lengths to inform model architecture decisions and batch processing strategies.
- 4) **Label Distribution Analysis:** Assesses class imbalance in intents and slot types, identifying rare classes that may require special handling.

- 5) BIO Tag Consistency: Validates that BIO tags follow proper syntactic rules (B-tags followed by I-tags of same type or O).

The reproducible exploration code implements these analyses:

```
from src.data.preprocess import load_atis_examples
import numpy as np
from collections import Counter

# Load all splits
train = load_atis_examples('data/raw/train.json')
dev = load_atis_examples('data/raw/dev.json')
test = load_atis_examples('data/raw/test.json')

# Basic counts
print(f'Train: {len(train)}, Dev: {len(dev)}, Test: {len(test)}')

# Intent analysis
all_intents = [ex['intent'] for ex in train]
intent_counts = Counter(all_intents)
print(f'Unique intents: {len(intent_counts)}')
print('Top 5 intents:', intent_counts.most_common(5))

# Slot analysis
all_slots = [slot for ex in train for slot in ex['slots']]
slot_counts = Counter(all_slots)
print(f'Unique slot labels: {len(slot_counts)}')
print('Slot distribution:', dict(slot_counts.most_common(10)))

# Sequence length analysis
train_lengths = [len(ex['tokens']) for ex in train]
print(f'Mean length: {np.mean(train_lengths):.1f}')
print(f'Median length: {np.median(train_lengths)}')
print(f'Max length: {max(train_lengths)}')
```

This systematic exploration reveals the ATIS dataset contains 4,978 training examples, 893 validation examples, and 893 test examples. The 21 unique intents range from common travel queries (atis_flight, atis_airfare) to specialized requests (atis_aircraft, atis_flight_time). The 120 BIO-formatted slot labels capture fine-grained semantic information, with 'O' (Outside) tags comprising approximately 65% of all tokens, creating a challenging class imbalance scenario for sequence labeling tasks.

2) Tokenization: methods, tradeoffs, and implementation considerations: Tokenization represents a critical preprocessing decision that directly impacts model performance, computational efficiency, and task complexity. For the ATIS dataset, we employ whitespace tokenization as the primary strategy, informed by careful analysis of the data characteristics and task requirements.

Whitespace Tokenization Rationale: The ATIS dataset exhibits several properties that make whitespace tokenization particularly suitable:

- 1) Data Quality: Utterances are relatively clean with

minimal punctuation and consistent spacing, reducing tokenization artifacts.

- 2) BIO Label Alignment: Direct correspondence between whitespace tokens and gold BIO labels simplifies the learning objective and avoids complex subword-to-word label aggregation.
- 3) Computational Efficiency: Extremely fast preprocessing with minimal computational overhead, enabling rapid experimentation.
- 4) Reproducibility: Deterministic tokenization across different environments and Python versions.
- 5) Interpretability: Token-level predictions directly correspond to human-readable words, facilitating error analysis and model debugging.

Technical Implementation Details: Our tokenizer implements case-insensitive processing (converting all text to lowercase) to reduce vocabulary size and improve generalization. Special handling is required for:

- Contractions: Words like "don't" are treated as single tokens to preserve semantic meaning.
- Numbers and Dates: Numeric sequences (e.g., "747", "11am") remain intact as they carry specific semantic information.
- Abbreviations: Common abbreviations like "BOS" (Boston airport code) are preserved.

Alternative Tokenization Strategies and Tradeoffs:

While whitespace tokenization serves our purposes effectively, several alternative approaches offer different advantages:

Subword Tokenization (WordPiece/SentencePiece):

- Advantages: Significantly reduces out-of-vocabulary (OOV) rates by decomposing rare words into frequent subword units. Enables use of pretrained Transformer tokenizers (BERT, RoBERTa) for transfer learning.
- Challenges: Requires complex label mapping strategies. For BIO tagging, the first subword inherits the B- or I- label, while subsequent subwords receive special continuation markers or are masked during evaluation.
- Implementation Complexity: Need to implement word-level aggregation for seqeval metrics, as subword predictions must be combined into word-level predictions for meaningful evaluation.

Character-level Tokenization:

- Advantages: Eliminates OOV entirely, handles morphological variations naturally.
- Challenges: Significantly increases sequence lengths and computational requirements. Requires CNN or specialized architectures to capture word-level semantics.

Morphological Tokenization:

- Advantages: Linguistically informed tokenization that respects word boundaries and morphological structure.